FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

AI-BASED PERSONALIZED COURSE RECOMMENDATION AND LEARNING PATH SYSTEM USING A* SEARCH

A PROJECT REPORT

Submitted by

2116240701502 – SHOBBIKA T

2116240701521 – SONA SREE R P



October 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled "AI-Based Personalized Course Recommendation and Learning Path System Using A* Search" is the bonafide work of Shobbika T (240701502) and Sona sree R P (240701521), who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____.

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

	ABSTRACT	
1	INTRODUCTION	
2	LITERATURE REVIEW	
3	PROPOSED SYSTEM	
	3.1 Architecture Diagram	
	3.2 A* Search Formulation	
4	MODULES DESCRIPTION	
	4.1 Student Profile and Input Module	
	4.2 Course Knowledge Base and Prerequisite Module	
	4.3 A* Search and Recommendation Module	
	4.4 Result Presentation Module	
5	IMPLEMENTATION AND RESULTS	
	5.1 Experimental Setup	
	5.2 Results and Testing	
6	CONCLUSION AND FUTURE WORK	
7	REFERENCES	

ABSTRACT

Choosing suitable courses for a career goal can be difficult because learners have different starting skills, levels of knowledge, available study time, and prerequisite requirements. A fixed course list may recommend material that is too advanced, repeat knowledge already acquired, or fail to provide the correct order for learning. This project proposes an AI-based personalized course recommendation and learning path system that uses the A* search algorithm to find a prerequisite-valid sequence of courses from a learner's current skills to the skills required for a selected career.

The system represents each search state as the set of courses completed so far. A course can be selected as a successor only when its prerequisites are satisfied. The accumulated cost g(n) represents the cost of courses selected along the current search path. In the initial implementation, a course cost is calculated from its estimated duration and difficulty. The heuristic h(n) estimates the remaining cost by examining missing career skills and the lowest available course cost associated with each missing skill; the maximum of these values is used as a lower-bound estimate. A* evaluates states using $f(n) = g(n) + h(n)$.

The proposed system accepts a learner's existing skills and career goal, consults a course knowledge base, searches for a suitable learning path, and presents the recommended sequence with its estimated cost. The project demonstrates an application of informed search from the Foundations of Artificial Intelligence syllabus. The current report describes the design and implementation approach. Actual execution screenshots, test outcomes, and measured results should be inserted after running the final version of the program.

CHAPTER 1: INTRODUCTION

1.1 Background

Online learning platforms provide a large number of courses, but learners may not know which course to take first or how individual courses connect to a long-term career objective. Course prerequisites, skill dependencies, difficulty, and duration make the selection problem more complex than simply sorting courses by popularity.

Artificial Intelligence (AI) includes problem-solving methods that search through possible states to find a path to a goal. Informed search algorithms use problem-specific estimates to guide the search. A* is an informed search algorithm that combines the cost already incurred with an estimate of the remaining cost.

1.2 Problem Statement

Students often face difficulties in selecting suitable courses and planning an effective learning path to achieve their desired career goals. Existing course selection approaches may not adequately consider a learner's current skills, course prerequisites, learning effort, and the skills required for a particular career. As a result, students may choose courses in an unsuitable order, repeat topics they already know, or struggle to identify the knowledge gaps they need to address.

The proposed AI-Based Personalized Course Recommendation and Learning Path System Using A Search* aims to address this problem by analyzing the learner's existing skills and selected career goal to recommend a suitable sequence of courses. The system uses the A* search algorithm to explore prerequisite-valid course paths and evaluate them based on course duration and difficulty. It also provides a structured study plan and supports progress tracking, helping students organize their learning and work systematically toward their career objectives.

1.3 Objectives

- Accept the learner's current skills and chosen career goal.

- Represent courses, skills, prerequisites, duration, and difficulty in a structured knowledge base.
- Use A* search to explore prerequisite-valid course sequences.
- Calculate path cost using g(n), estimate remaining cost using h(n), and rank states using f(n) = g(n) + h(n).
- Display a course sequence that can help the learner acquire the skills required for the target career.
- Demonstrate a practical application of informed search without requiring a machine-learning model.

1.4 Scope

The project focuses on rule-based course planning using A* search. Recommendation quality depends on the completeness and accuracy of the course catalogue, skill mappings, prerequisites, duration estimates, and career requirements. The system does not claim to predict student success or replace professional academic guidance.

1.5 Organization of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 reviews related approaches. Chapter 3 explains the proposed architecture and A* formulation. Chapter 4 describes the system modules. Chapter 5 records the implementation and evaluation procedure. Chapter 6 presents the conclusion and possible extensions, followed by references.

CHAPTER 2: LITERATURE REVIEW

Course recommendation systems commonly use content-based filtering, collaborative filtering, knowledge-based rules, or hybrid approaches. Content-based systems compare course attributes with a learner profile. Collaborative filtering uses patterns in the choices or ratings of other learners. Knowledge-based systems use explicit requirements and constraints, such as prerequisites and career skills.

For a project focused on search algorithms, a knowledge-based approach is appropriate because course prerequisites and career requirements can be represented explicitly. The problem can be treated as a state-space search: a state records the courses already completed, an action adds an eligible course, and a goal test checks whether the required career skills have been acquired.

Uniform-cost search expands states in increasing accumulated cost order. Greedy best-first search prioritizes an estimate of distance to the goal. A* combines both ideas by evaluating $f(n) = g(n) + h(n)$. With non-negative step costs and an appropriate admissible heuristic, A* can find a least-cost solution under the conditions of the search formulation. Its performance still depends on the size of the state space and the quality of the heuristic.

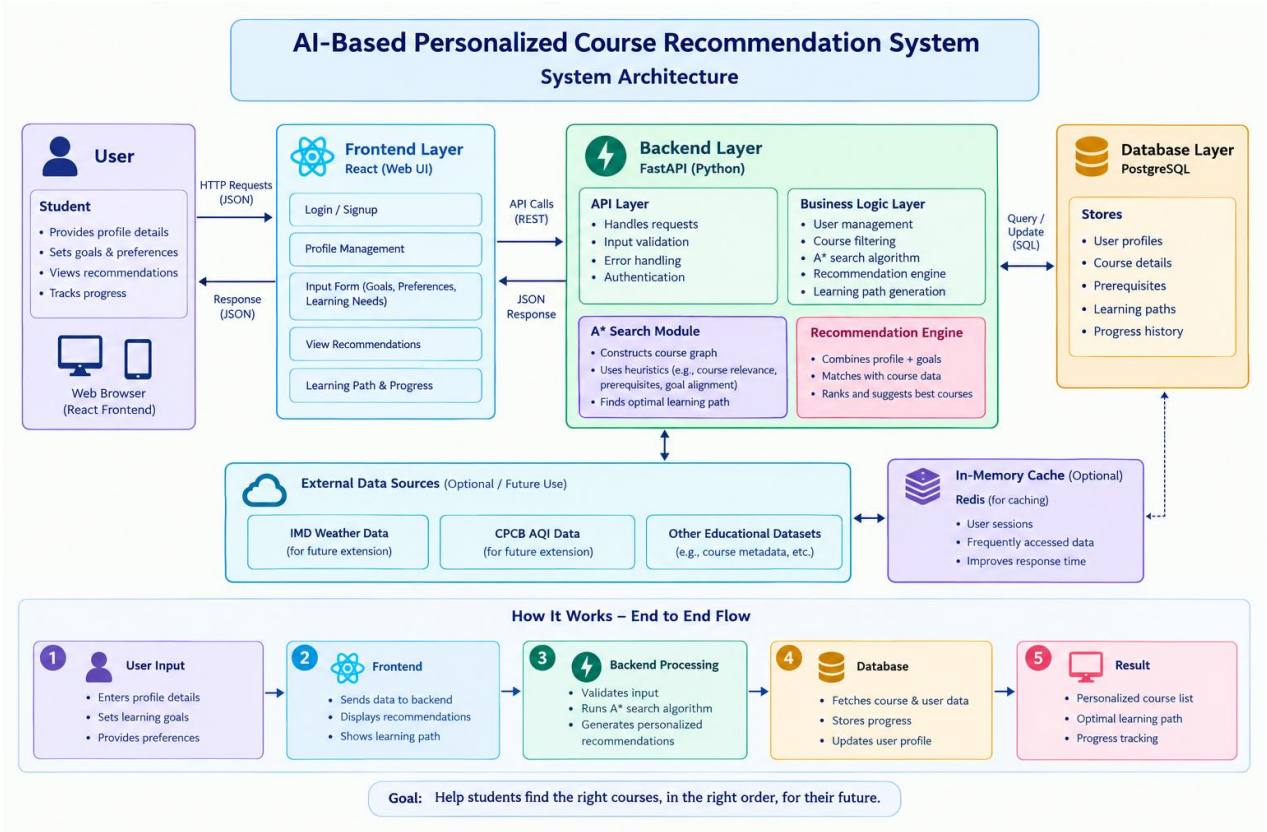
Unlike systems that require a trained machine-learning model or user-rating history, the proposed prototype uses a structured course catalogue and explicit search logic. This makes the decision process easier to explain in a classroom demonstration. The quality of its recommendations is limited by the data supplied and by the cost and heuristic design.

CHAPTER 3: PROPOSED SYSTEM

3.1 System Overview

The proposed system receives student information, determines which skills are already available, loads course and career data, and applies A* search to find a course sequence that satisfies the selected career's skill requirements. Each action in the search adds one eligible course to the completed-course set.

3.2 Architecture Diagram



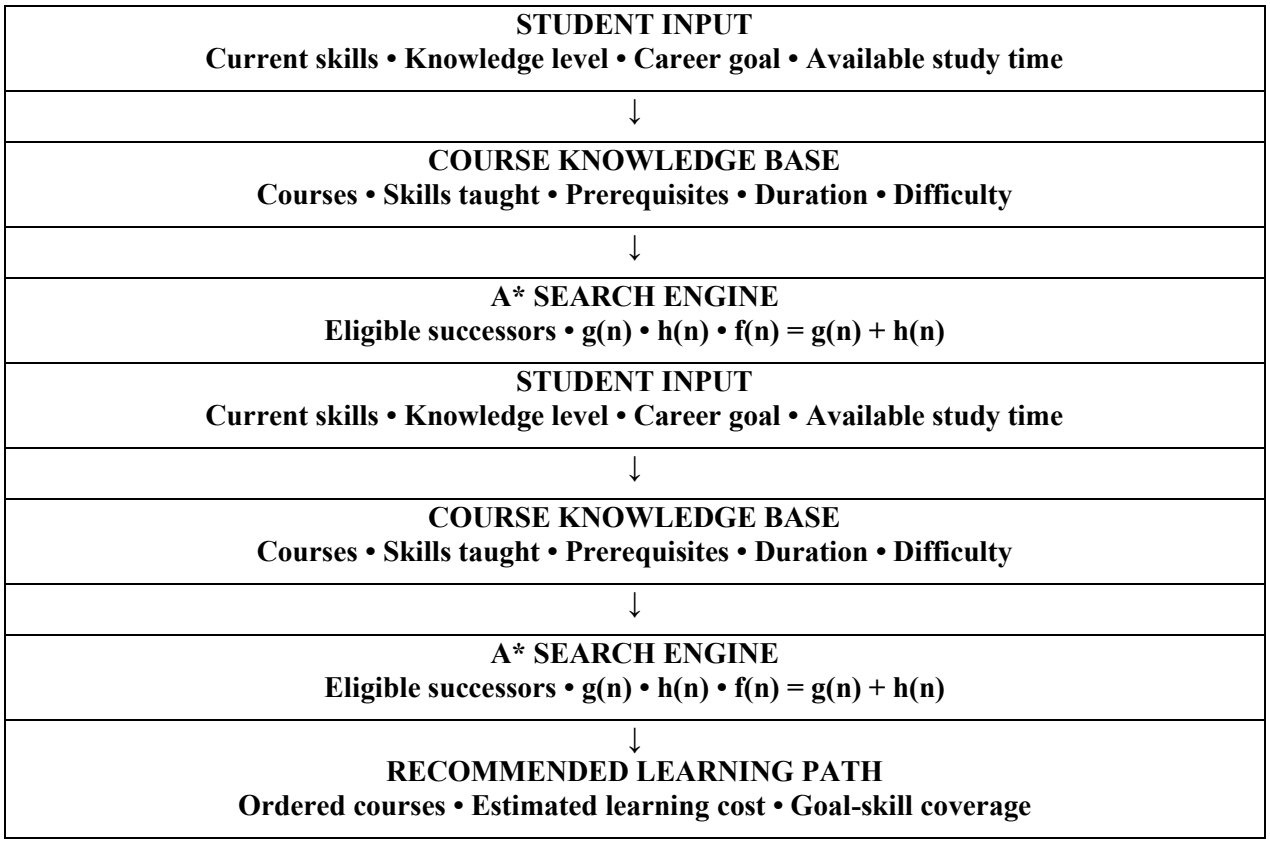


Figure 3.1: High-level architecture of the proposed system.

3.3 State Representation

A search state is represented by a set of completed course identifiers. A frozenset or equivalent immutable representation can be used in Python so that states can be stored in search data structures. The learner's initial skills are retained separately. The skills available in a state are the union of the learner's initial skills and the skills taught by courses included in that state.

3.4 Successor Generation and Goal Test

To generate successors, the system checks each course that has not yet been selected. A course is eligible only if all its prerequisite skills are available in the current state. Selecting an eligible course creates a new state with that course added. The goal test succeeds when the skills available in the state cover all skills required by the chosen career.

3.5 A* Search Formulation

A* evaluates each state n using f(n) = g(n) + h(n). The implementation uses the following definitions:

- g(n) — accumulated cost of courses selected so far. Each course cost is calculated as duration_hours + (difficulty − 1) × 10.
- h(n) — estimated remaining cost to acquire missing career skills. The prototype finds the minimum course cost associated with each missing required skill and uses the maximum of those minimum costs.
- f(n) — estimated total cost through the current state, calculated as g(n) + h(n). The state with the lowest f(n) is prioritized for expansion.

The difficulty adjustment is a design choice for the prototype, not a universal measure of learning effort. Duration and difficulty values should be calibrated and documented. The heuristic's admissibility depends on the course data and skill coverage assumptions; if a course teaches multiple missing skills, or prerequisites create additional cost, the implementation should be evaluated carefully before claiming a formal optimality guarantee.

3.6 Cost Function

For a course with estimated duration D hours and difficulty rating L, where L is an integer on the chosen difficulty scale, the prototype uses: CourseCost = D + (L − 1) × 10. The path cost g(n) is the sum of the individual course costs for the courses selected along the path. The formula gives more difficult courses an additional penalty.

3.7 Personalization

The initial recommendation is personalized primarily through the learner's existing skills and career goal. Courses that teach skills already possessed may be unnecessary unless they are needed for another reason. Additional preferences such as interests, weekly study hours, language, budget, course format, and preferred difficulty can be incorporated as constraints or tie-breaking factors in future versions.

CHAPTER 4: MODULES DESCRIPTION

4.1 Student Profile and Input Module

This module collects the learner's existing skills and selected career goal. The initial skill list forms the starting knowledge state. Input validation should ensure that the selected career exists in the catalogue and that skill names match the knowledge-base identifiers. A user interface can later add knowledge level, interests, and weekly availability.

4.2 Course Knowledge Base and Prerequisite Module

The knowledge base stores course identifiers, titles, skills taught, prerequisite skills, duration estimates, and difficulty values. Career records store the skills required for each goal. A JSON file is suitable for a small prototype because it is easy to inspect and edit. A larger application may use SQLite or another database. The data should be reviewed by a subject expert before being treated as an authoritative curriculum.

4.3 A* Search and Recommendation Module

This is the main AI module. It maintains a frontier of states awaiting exploration, records the best known cost to reach each state, and expands the state with the lowest f(n). It generates only courses whose prerequisites are satisfied, updates the accumulated cost when a course is added, and checks whether the career goal has been reached. If no valid sequence is found, the program should report that no path was found in the current knowledge base rather than inventing a recommendation.

4.4 Result Presentation Module

This module displays the recommended sequence in order, along with course titles and cost estimates. A useful result screen should also show the selected career goal, skills already known, skills still required, and a clear message if the available course data cannot satisfy the goal. The explanation should make clear that results depend on the course catalogue and the selected cost model.

4.5 Error Handling and Validation

- The Error Handling and Validation Module helps ensure that the system processes valid inputs and consistent course data. It identifies errors that could affect the correctness of the recommendation process or the stability of the application.
- The project incorporates dataset validation and API-level request handling to manage common problems. Its main responsibilities include:
  - Validating course and career data before using it for search.
  - Detecting duplicate course identifiers and invalid course attributes.
  - Checking prerequisite and skill references for consistency.
  - Handling invalid career selections and unsuccessful recommendation requests.
  - Avoiding repeated exploration of search states when a better cost is already known.
  - Reporting when no valid learning path is available.
  - Returning appropriate error responses when API requests cannot be processed.
- These validation mechanisms improve reliability and make it easier to identify problems in the input data or recommendation process.

CHAPTER 5: IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The proposed system is implemented as a web-based application that provides personalized course recommendations and learning paths based on the learner's skills, career goals, and learning preferences. The application follows a client-server architecture, where the frontend collects user information and displays recommendations, while the backend processes requests and executes the recommendation logic.

The frontend is developed using React, which provides an interactive user interface for entering learner details, viewing course recommendations, and accessing learning-path information. The backend is developed using Python and FastAPI, which handles API requests, validates input data, processes recommendation requests, and communicates with the database. SQLite is used to store and retrieve the application data, according to the project configuration.

The A* search algorithm is the main Artificial Intelligence technique used to identify a suitable sequence of courses. It evaluates possible course paths using the evaluation function:

[f(n)=g(n)+h(n)]

Here, (g(n)) represents the accumulated cost of the courses selected so far, while (h(n)) estimates the remaining cost of reaching the target career requirements. The search process uses course prerequisites to determine which courses can be selected at each stage.

The system can be tested on a computer with Python, Node.js, a web browser, and the required project dependencies installed. The backend and frontend are started separately unless the provided startup script is configured to launch both services. The application is then accessed through the local frontend address.

The following details should be recorded during the final experiment:

- Programming language: Python and JavaScript.
- Frontend framework: React.
- Backend framework: FastAPI.
- Database: SQLite.
- AI technique: A* search algorithm.
- Testing environment: Record the actual operating system and software versions.
- Dataset: Record the actual number of courses, career goals, prerequisite relationships, and skill mappings in the submitted project.

The experiments should be performed using different learner profiles and career goals to determine whether the system generates valid, prerequisite-aware course recommendations.

5.2 Test Plan

Test case	Input / condition	Expected behaviour
T1: Valid goal	Known career goal and a valid starting skill set	Return a prerequisite-valid path if one exists.
T2: Skills already known	Starting skills cover all career requirements	Return an empty or zero-additional-course path, according to implementation design.
T3: Missing prerequisite	A candidate course has a prerequisite not yet satisfied	Do not select that course until prerequisites are met.
T4: Unknown career	Career identifier is absent from the knowledge base	Show a clear validation message.
T5: Unreachable goal	No available courses cover a required skill	Report that no valid path was found.
T6: Search correctness	Small hand-checkable course graph	Returned path reaches the goal and its accumulated cost matches the selected courses.

5.3 Results

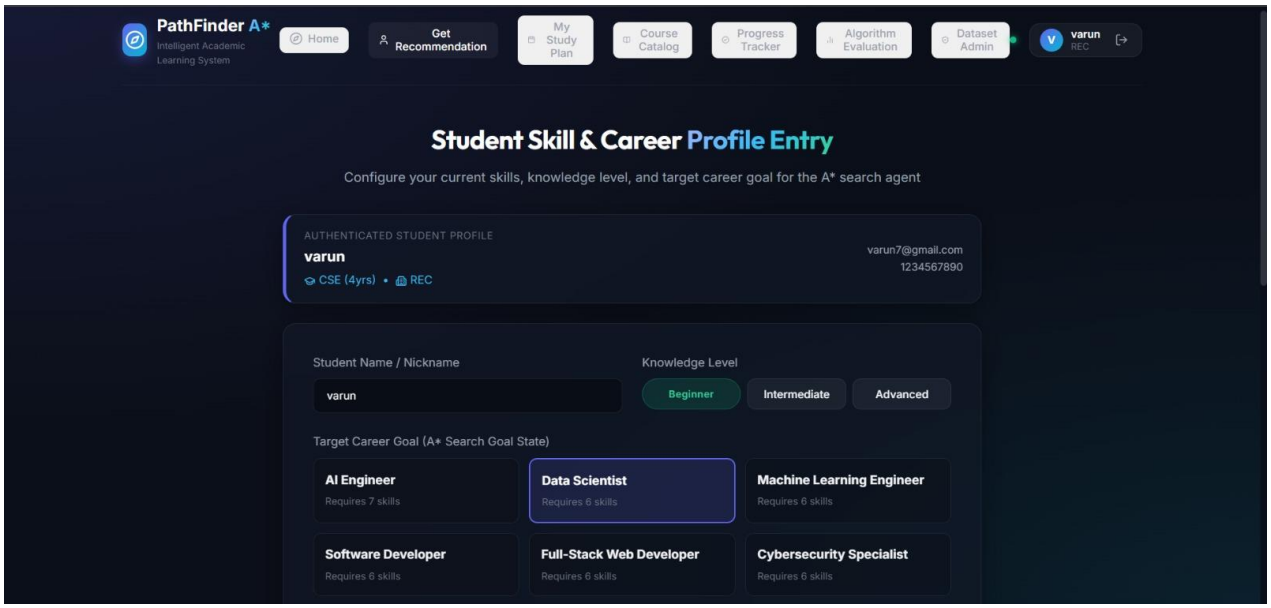
The application is designed to produce a personalized sequence of courses based on the learner's initial skills and selected career goal. The recommendation engine uses A* search to explore possible course combinations and prioritize states according to their accumulated cost and estimated remaining cost.

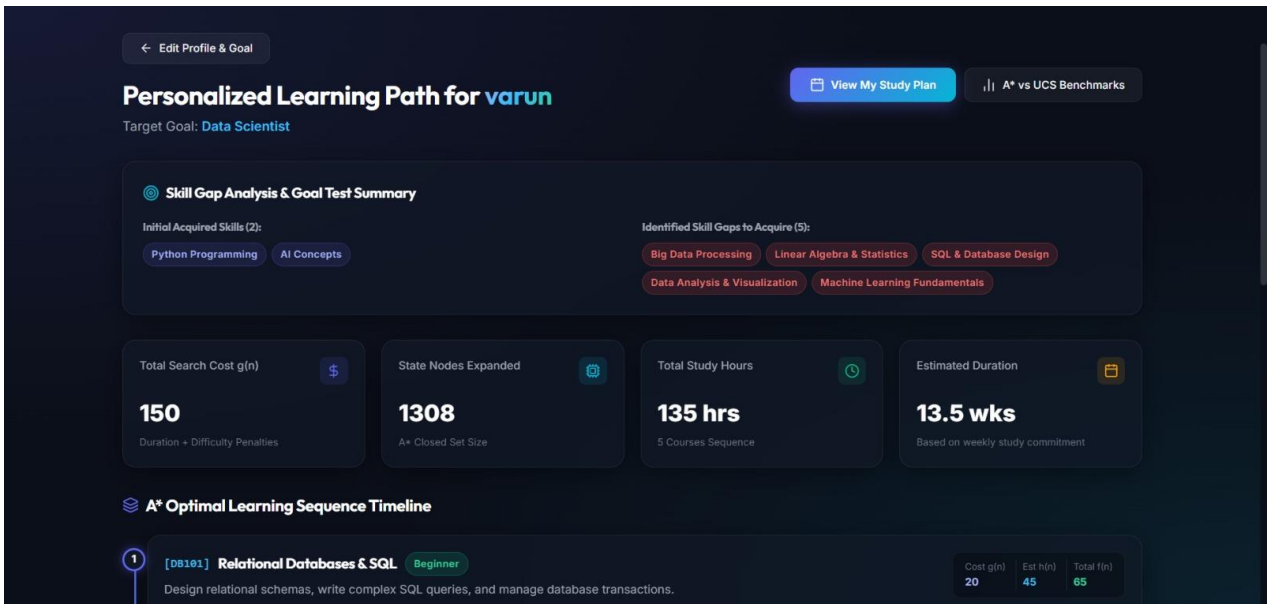
The course knowledge base contains information about courses, their associated skills, prerequisites, duration, and difficulty. During the search process, courses that do not satisfy their prerequisite requirements are excluded from the available successor states. This helps ensure that the generated learning path follows a meaningful learning sequence.

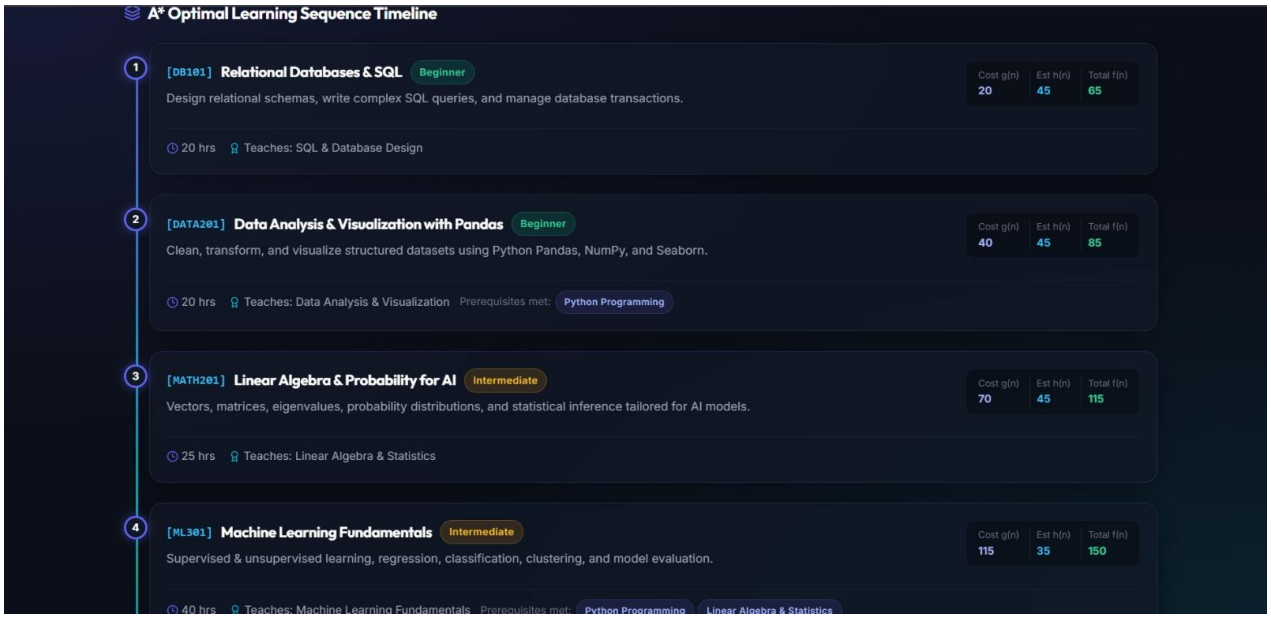
The implementation also considers skills that the learner already possesses. For example, when the learner selects AI Engineer as a career goal and indicates that Python Programming is already known, the system can avoid recommending the introductory Python course. This reduces unnecessary learning and can lower the additional cost of the recommended path.

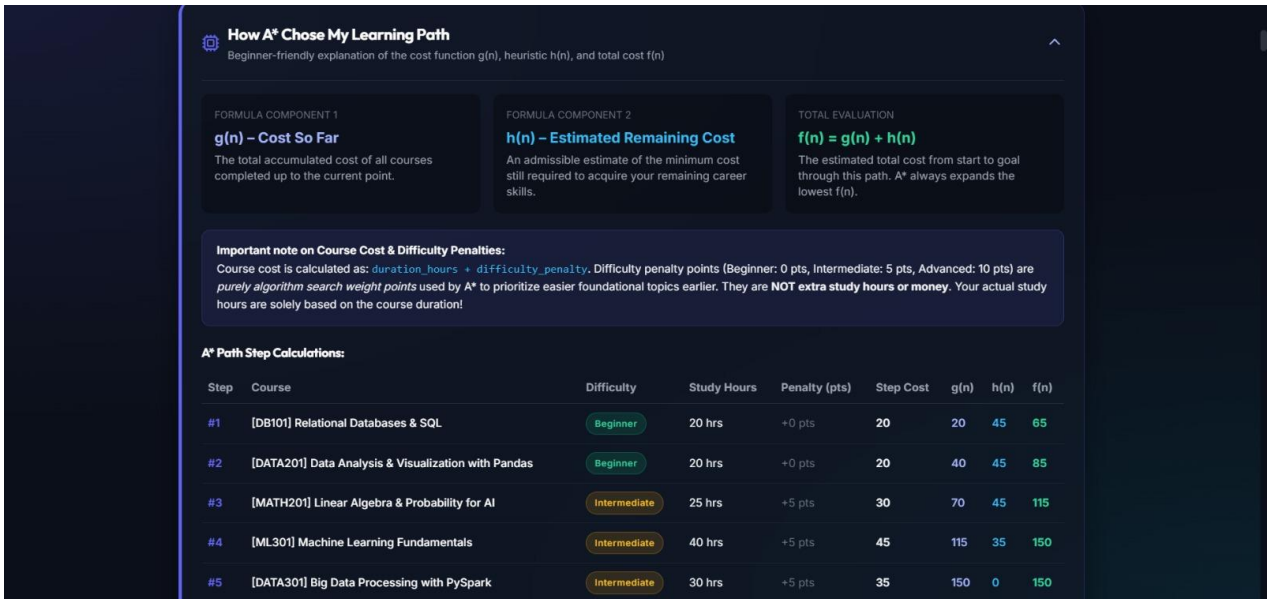
The project includes a Uniform Cost Search implementation for comparison with A*. Both algorithms can be evaluated using the same initial skills and career goal. The comparison considers:

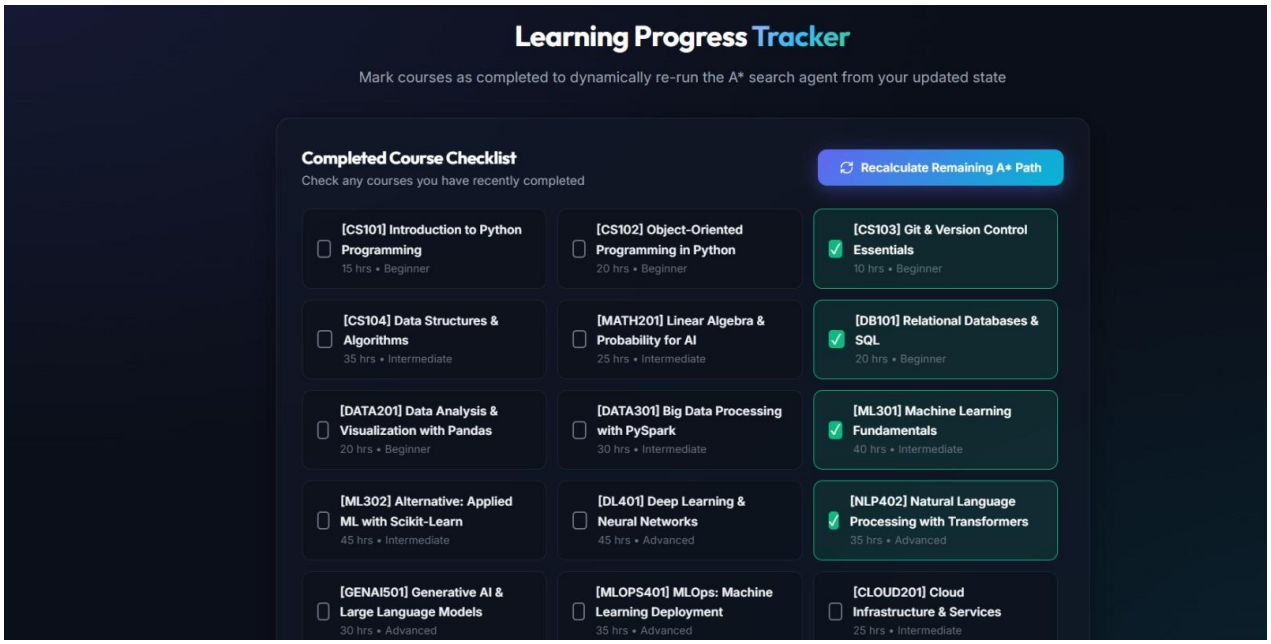
- Total cost of the generated learning path.
- Total course duration.
- Number of search nodes expanded.
- Algorithm execution time.
- Whether both algorithms return the same least-cost solution.


PathFinder A*
Intelligent Academic Learning System
Home
Get Recommendation
My Study Plan
Course Catalog
Progress Tracker
Algorithm Evaluation
Dataset Admin
varun
REC
Student Skill & Career Profile Entry
Configure your current skills, knowledge level, and target career goal for the A* search agent
AUTHENTICATED STUDENT PROFILE
varun
CSE (4yrs) • REC
varun7@gmail.com
1234567890
Student Name / Nickname
varun
Knowledge Level
Beginner
Intermediate
Advanced
Target Career Goal (A* Search Goal State)
AI Engineer
Requires 7 skills
Data Scientist
Requires 6 skills
Machine Learning Engineer
Requires 6 skills
Software Developer
Requires 6 skills
Full-Stack Web Developer
Requires 6 skills
Cybersecurity Specialist
Requires 6 skills


Edit Profile & Goal
Personalized Learning Path for varun
Target Goal: Data Scientist
View My Study Plan
A* vs UCS Benchmarks
Skill Gap Analysis & Goal Test Summary
Initial Acquired Skills (2):
Python Programming
AI Concepts
Identified Skill Gaps to Acquire (5):
Big Data Processing
Linear Algebra & Statistics
SQL & Database Design
Data Analysis & Visualization
Machine Learning Fundamentals
Total Search Cost g(n)
150
Duration + Difficulty Penalties
State Nodes Expanded
1308
A* Closed Set Size
Total Study Hours
135 hrs
5 Courses Sequence
Estimated Duration
13.5 wks
Based on weekly study commitment
A* Optimal Learning Sequence Timeline
1
[DB101] Relational Databases & SQL
Beginner
Design relational schemas, write complex SQL queries, and manage database transactions.
Cost g(n) 20
Est h(n) 45
Total f(n) 65


A* Optimal Learning Sequence Timeline
1
[DB101] Relational Databases & SQL
Beginner
Design relational schemas, write complex SQL queries, and manage database transactions.
Cost g(n) 20
Est h(n) 45
Total f(n) 65
20 hrs
Teaches: SQL & Database Design
2
[DATA201] Data Analysis & Visualization with Pandas
Beginner
Clean, transform, and visualize structured datasets using Python Pandas, NumPy, and Seaborn.
Cost g(n) 40
Est h(n) 45
Total f(n) 85
20 hrs
Teaches: Data Analysis & Visualization
Prerequisites met:
Python Programming
3
[MATH201] Linear Algebra & Probability for AI
Intermediate
Vectors, matrices, eigenvalues, probability distributions, and statistical inference tailored for AI models.
Cost g(n) 70
Est h(n) 45
Total f(n) 115
25 hrs
Teaches: Linear Algebra & Statistics
4
[ML301] Machine Learning Fundamentals
Intermediate
Supervised & unsupervised learning, regression, classification, clustering, and model evaluation.
Cost g(n) 115
Est h(n) 35
Total f(n) 150
40 hrs
Teaches: Machine Learning Fundamentals
Prerequisites met:
Python Programming
Linear Algebra & Statistics


How A* Chose My Learning Path
Beginner-friendly explanation of the cost function g(n), heuristic h(n), and total cost f(n)
FORMULA COMPONENT 1
g(n) – Cost So Far
The total accumulated cost of all courses completed up to the current point.
FORMULA COMPONENT 2
h(n) – Estimated Remaining Cost
An admissible estimate of the minimum cost still required to acquire your remaining career skills.
TOTAL EVALUATION
f(n) = g(n) + h(n)
The estimated total cost from start to goal through this path. A* always expands the lowest f(n).
Important note on Course Cost & Difficulty Penalties:
Course cost is calculated as: duration_hours + difficulty_penalty. Difficulty penalty points (Beginner: 0 pts, Intermediate: 5 pts, Advanced: 10 pts) are purely algorithm search weight points used by A* to prioritize easier foundational topics earlier. They are NOT extra study hours or money. Your actual study hours are solely based on the course duration!
A* Path Step Calculations:
Step Course Difficulty Study Hours Penalty (pts) Step Cost g(n) h(n) f(n)
#1 [DB101] Relational Databases & SQL Beginner 20 hrs +0 pts 20 20 45 65
#2 [DATA201] Data Analysis & Visualization with Pandas Beginner 20 hrs +0 pts 20 40 45 85
#3 [MATH201] Linear Algebra & Probability for AI Intermediate 25 hrs +5 pts 30 70 45 115
#4 [ML301] Machine Learning Fundamentals Intermediate 40 hrs +5 pts 45 115 35 150
#5 [DATA301] Big Data Processing with PySpark Intermediate 30 hrs +5 pts 35 150 0 150


Learning Progress Tracker
Mark courses as completed to dynamically re-run the A* search agent from your updated state
Completed Course Checklist
Check any courses you have recently completed
Recalculate Remaining A* Path
[CS101] Introduction to Python Programming
15 hrs • Beginner
[CS102] Object-Oriented Programming in Python
20 hrs • Beginner
[CS103] Git & Version Control Essentials
10 hrs • Beginner
[CS104] Data Structures & Algorithms
35 hrs • Intermediate
[MATH201] Linear Algebra & Probability for AI
25 hrs • Intermediate
[DB101] Relational Databases & SQL
20 hrs • Beginner
[DATA201] Data Analysis & Visualization with Pandas
20 hrs • Beginner
[DATA301] Big Data Processing with PySpark
30 hrs • Intermediate
[ML301] Machine Learning Fundamentals
40 hrs • Intermediate
[ML302] Alternative: Applied ML with Scikit-Learn
45 hrs • Intermediate
[DL401] Deep Learning & Neural Networks
45 hrs • Advanced
[NLP402] Natural Language Processing with Transformers
35 hrs • Advanced
[GENAI501] Generative AI & Large Language Models
30 hrs • Advanced
[MLOPS401] MLOps: Machine Learning Deployment
35 hrs • Advanced
[CLOUD201] Cloud Infrastructure & Services
25 hrs • Intermediate

5.4 Evaluation Criteria

The performance and correctness of the proposed system are evaluated using the following criteria.

1. Goal coverage

The recommended learning path should enable the learner to acquire all skills required by the selected career goal. The final acquired skill set is compared against the career's required skills.

2. Prerequisite validity

Every recommended course must satisfy its prerequisite requirements before it is selected. This ensures that advanced courses are not recommended before the learner has the necessary foundational knowledge.

3. Cost consistency

The total path cost returned by the recommendation engine should equal the sum of the individual costs of the courses in the path. The project uses the following cost function:

[C(c)=D(c)+P(c)]

where (D(c)) is the course duration in hours and (P(c)) is the difficulty penalty. In the submitted implementation, the difficulty penalties are 0 for Beginner, 5 for Intermediate, and 10 for Advanced courses.

4. Search correctness

A* results are compared with UCS results for identical initial skills and career goals. Matching solution costs provide evidence that A* is finding a least-cost solution for the tested scenarios. The project's automated tests also check path validity and cost consistency.

5. Search efficiency

The number of nodes expanded and the execution time are recorded for both algorithms. Fewer expanded nodes may indicate that the heuristic helps A* focus its search, although the actual result depends on the dataset and search scenario.

6. Input validation and robustness

The system should handle unknown career identifiers, unreachable goals, and invalid or incomplete inputs with understandable error messages rather than silently generating an invalid learning path.

7. Study plan consistency

The generated study plan should respect the planner's scheduling constraints, including available study hours and the order in which courses can be completed. Progress information should be updated consistently when learners mark tasks as completed.

5.5 Limitations

The output quality depends on the course knowledge base and the accuracy of prerequisite and skill mappings. The duration and difficulty score is a simplified cost model. The heuristic should be validated against the formal course graph before making a strong optimality claim. The current design does not automatically learn from student feedback and does not independently verify external course quality, availability, or certification.

CHAPTER 6: CONCLUSION AND FUTURE WORK

This project presents an AI-based personalized course recommendation and learning path system using A* search. It models course planning as a state-space search problem, represents the completed courses as a state, checks prerequisites before generating successors, and tests whether the learner's skills cover the selected career requirements. The accumulated cost g(n) and estimated remaining cost h(n) are combined as f(n) = g(n) + h(n) to guide the search.

The project demonstrates how an informed search algorithm from the Foundations of Artificial Intelligence syllabus can be applied to a practical learning-planning problem. Before final submission, the implementation should be run and tested, screenshots should be added, and the report's environment and results sections should be updated with observed evidence.

6.1 Future Work

- Build a web interface with a student profile, course details, and progress tracking.
- Expand and validate the course knowledge base with reliable course descriptions and prerequisites.
- Add interests, weekly study time, learning format, and language preferences.
- Use a database and provide an interface for maintaining courses and career requirements.
- Compare A* against uniform-cost search on controlled test cases and record path cost and number of expanded states.
- Improve heuristic design and document the conditions under which it is admissible.
- Allow learners to mark courses complete and regenerate the remaining path from their updated skills.

CHAPTER 7: REFERENCES

- Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136.

- Poole, D. and Mackworth, A. (2017). Artificial Intelligence: Foundations of Computational Agents. 2nd ed. Cambridge University Press.

- Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/ (accessed 19 October 2026).

- Ricci, F., Rokach, L., and Shapira, B. (2022). "Recommender Systems: Introduction and Challenges." In Recommender Systems Handbook. Springer. https://doi.org/10.1007/978-1-4899-7637-6_1.

- Rich, E., Knight, K. and Nair, S. B. (2009). Artificial Intelligence. 3rd ed. Tata McGraw-Hill.

- Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach. 4th ed. Pearson. Official publisher page.

- Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach. 4th ed. Pearson.